

Dossier de remise - MyWorkoutApp

1. Rappel du sujet

Nom du projet : MyWorkoutApp

MyWorkoutApp est une application mobile développée en React Native avec Expo. Son objectif est de proposer un suivi d'entraînement simple, rapide et utilisable directement sur téléphone pendant une séance de musculation.

L'idée principale du projet est de réduire au maximum la friction pendant l'utilisation. L'utilisateur doit pouvoir créer une séance, ajouter ses exercices, enregistrer ses séries et retrouver facilement son historique, sans dépendre d'un serveur externe ni d'une connexion Internet.

L'application permet donc de :

- créer une séance,
- encoder une durée estimée,
- ajouter un commentaire,
- préparer une liste d'exercices,
- ajouter des séries avec poids et répétitions,
- marquer les séries comme complétées,
- consulter l'historique des séances enregistrées.

Le projet reste volontairement centré sur les fonctionnalités essentielles demandées dans le cadre du cours. Il ne cherche pas à couvrir tous les besoins possibles d'une application de sport complète, mais à proposer une solution claire, cohérente et techniquement maîtrisée.

2. Spécifications des choix

Les fonctionnalités effectivement implémentées dans l'application sont les suivantes :

- navigation principale avec Expo Router ;
- onglet Aujourd'hui pour préparer une nouvelle séance ;
- onglet Archives pour consulter les séances déjà enregistrées ;
- écran dynamique de détail d'une séance à partir de son identifiant ;
- création d'une séance avec durée, commentaire et liste d'exercices ;
- validation des formulaires avec React Hook Form et Zod ;
- ajout dynamique de plusieurs exercices dans le formulaire de création ;
- stockage local des données dans SQLite ;
- base relationnelle composée des tables sessions, exercices et sets ;
- ajout d'exercices supplémentaires dans une séance existante ;
- ajout de séries avec poids et nombre de répétitions ;

- marquage d'une série comme complétée ;
- retour haptique lors de la complétion d'une série ;
- calcul et affichage d'informations simples dans l'historique : nombre d'exercices, nombre de séries, séries complétées, volume total et volume moyen ;
- affichage d'états de chargement avec `ActivityIndicator` ;
- affichage d'états vides avec un composant dédié ;
- gestion d'erreurs simples avec message et bouton de réessai ;
- rafraîchissement manuel des listes et du détail d'une séance ;
- gestion du focus clavier entre champs pour accélérer la saisie.

3. Justification technique

React Native et Expo

Le projet a été réalisé en React Native avec Expo parce qu'il s'agit de la stack utilisée dans le cours. Ce choix permet de rester cohérent avec la matière vue et d'utiliser un environnement de développement bien adapté à un projet mobile.

Expo simplifie plusieurs points importants :

- lancement rapide du projet ;
- test sur simulateur iOS ;
- test sur iPhone avec Expo Go ;
- intégration plus simple de bibliothèques comme expo-sqlite et expo-haptics.

Dans le cadre d'un projet académique, cette solution est pertinente car elle permet de se concentrer sur la logique applicative plutôt que sur la configuration native.

TypeScript

TypeScript a été utilisé avec un typage strict afin d'éviter les erreurs simples et de rendre le code plus lisible. Le typage est utilisé pour :

- les données manipulées dans SQLite ;
- les propriétés des composants ;
- les valeurs de formulaires ;
- les données retournées par les hooks personnalisés.

L'utilisation de TypeScript permet aussi de garder une cohérence entre la validation Zod et les types réellement utilisés dans l'application.

Expo Router

Expo Router a été choisi car c'est la solution de navigation vue au cours. La navigation basée sur les fichiers rend la structure du projet plus claire :

- un layout racine ;

- un groupe d'onglets pour les écrans principaux ;
- une route dynamique pour le détail d'une séance.

Ce choix facilite la lecture du projet et évite d'ajouter une configuration de navigation plus lourde que nécessaire.

SQLite

SQLite a été retenu car les données du projet sont relationnelles :

- une séance possède plusieurs exercices ;
- un exercice possède plusieurs séries.

Ce type de structure est mieux adapté à une base relationnelle qu'à AsyncStorage. De plus, le sujet ne demande pas de synchronisation distante. Un stockage local est donc suffisant, plus simple et plus cohérent avec l'objectif de rapidité.

Le projet utilise aussi plusieurs éléments importants vus au cours :

- SQLiteProvider à la racine de l'application ;
- useSQLiteContext pour accéder à la base dans les hooks ;
- migrations gérées avec PRAGMA user_version ;
- activation de foreign_keys pour garantir l'intégrité relationnelle ;
- activation du mode WAL pour un fonctionnement plus fiable ;
- booléens stockés sous forme d'entiers 0 et 1 ;
- dates stockées sous forme de texte.

Transactions et cohérence des données

Certaines opérations sont regroupées dans des transactions exclusives. C'est le cas lors de la création d'une séance avec ses exercices, ainsi que lors de l'insertion d'un lot de séries.

Ce choix est important pour garantir la cohérence des données. Si une erreur se produit pendant l'opération, on évite qu'une partie seulement des données soit enregistrée.

Pour l'ajout de plusieurs séries, une requête préparée est également utilisée afin d'éviter de recréer la requête SQL à chaque insertion.

React Hook Form et Zod

React Hook Form et Zod ont été choisis car ces outils sont explicitement vus dans le cours et adaptés aux formulaires mobiles.

React Hook Form gère :

- l'état du formulaire ;
- les champs contrôlés ;
- la soumission ;

- l'intégration avec les composants React Native.

Zod gère :

- les règles de validation ;
- les messages d'erreur ;
- l'inférence de types à partir des schémas.

Ce choix permet de centraliser la validation et d'éviter une logique dispersée dans les composants.

Gestion d'état

La gestion d'état a été volontairement gardée simple. Le projet repose principalement sur :

- useState ;
- useEffect ;
- useRef ;
- quelques hooks personnalisés pour isoler l'accès aux données SQLite.

Il n'y a pas de bibliothèque externe de state management, car cela aurait ajouté une complexité inutile au regard de la taille du projet. Les besoins restent locaux aux écrans et peuvent être gérés correctement avec les outils de base de React.

Axios et Fetch

Le projet n'utilise ni Axios ni Fetch. Ce choix est logique car l'application ne communique avec aucune API distante. Toutes les données sont enregistrées localement dans SQLite.

Ajouter une bibliothèque réseau n'aurait donc eu aucun intérêt fonctionnel et aurait introduit une dépendance inutile.

Haptics

La bibliothèque expo-haptics est utilisée lors de la complétion d'une série. Ce retour tactile reste simple, mais il améliore l'expérience utilisateur sur mobile en confirmant immédiatement l'action réalisée.

Ce choix est cohérent avec l'objectif du projet : rendre l'encodage pendant une séance rapide et naturel.

4. Architecture

Structure des dossiers

L'architecture du projet suit la logique vue au cours et cherche à séparer clairement la navigation, l'interface, l'accès aux données et la configuration.

- app/ : contient les routes Expo Router ;
- app/(tabs)/ : contient les deux écrans principaux, Aujourd'hui et Archives ;
- app/session/[id].tsx : contient l'écran de détail d'une séance ;

- `src/components/` : contient les composants d'interface réutilisables ;
- `src/constants/` : contient les constantes visuelles partagées ;
- `src/context/` : contient le provider SQLite ;
- `src/db/` : contient les migrations et les types liés à la base de données ;
- `src/hooks/` : contient la logique d'accès aux données et les opérations métier.

Cette organisation permet d'éviter de mélanger dans un même fichier :

- la navigation ;
- l'interface ;
- la validation ;
- les requêtes SQLite.

Flux de données

Le fonctionnement général de l'application repose sur un flux simple : l'écran appelle un hook, le hook interagit avec SQLite, puis l'écran se met à jour avec les nouvelles données.

Création d'une séance

1. L'utilisateur remplit le formulaire sur l'écran Aujourd'hui.
2. React Hook Form récupère les valeurs introduites.
3. Zod valide les données avant l'enregistrement.
4. Le hook de création enregistre la séance dans SQLite.
5. La séance et les exercices sont insérés dans une transaction exclusive.
6. L'identifiant de la nouvelle séance est récupéré.
7. L'utilisateur est redirigé vers l'écran de détail correspondant.

Consultation du détail d'une séance

1. L'identifiant est lu depuis la route dynamique.
2. Un hook charge la séance, ses exercices et ses séries depuis SQLite.
3. Les données sont transformées en structure exploitable par l'écran.
4. L'utilisateur peut ajouter un exercice ou des séries.
5. Les opérations d'ajout ou de mise à jour passent par un hook de mutations.
6. Après modification, l'écran recharge les données pour afficher l'état à jour.

Consultation de l'historique

1. Un hook charge les séances depuis SQLite.
2. Une requête SQL regroupe les données nécessaires à l'affichage.
3. Les informations calculées incluent le nombre d'exercices, le nombre de séries, le nombre de séries complétées et le volume total.

4. Les résultats sont affichés dans une FlatList.

5. Auto-évaluation

Le tableau suivant compare les fonctionnalités prévues avec ce qui a réellement été implémenté dans le projet.

Fonctionnalité annoncée	Réalisée	Commentaire
Navigation avec Expo Router, onglets et routes dynamiques	Oui	Écrans Aujourd'hui, Archives et détail d'une séance
Création d'une séance	Oui	Formulaire validé avec React Hook Form et Zod
Ajout de plusieurs exercices	Oui	Ajout dynamique dans le formulaire de création
Stockage local SQLite	Oui	Base relationnelle locale
Migrations SQLite	Oui	Initialisation avec PRAGMA user_version
Ajout de séries	Oui	Poids et répétitions enregistrés par exercice
Marquage des séries comme complétées	Oui	Mise à jour depuis l'écran de détail
Retour haptique	Oui	Déclenché lors de la complétion d'une série
Historique des séances	Oui	Liste des séances enregistrées avec résumé
Statistiques simples	Oui	Volume total, volume moyen et progression
États de chargement, erreurs et réessai	Oui	Présents sur les écrans principaux
Rafraîchissement manuel	Oui	Sur l'historique et sur le détail d'une séance
Synchronisation serveur	Non	Non demandée dans le sujet
Authentification utilisateur	Non	Hors périmètre du projet
Préférences avancées	Non	Non prioritaire pour cette version

L'ensemble des objectifs principaux du sujet a donc été atteint. Les éléments non réalisés correspondent à des fonctionnalités volontairement laissées hors périmètre afin de conserver un projet simple, cohérent et conforme aux attentes du cours.

Conclusion

MyWorkoutApp répond aux objectifs fixés au départ : permettre un suivi d'entraînement local, rapide et simple à utiliser sur mobile.

Le projet reste volontairement raisonnable dans son niveau de complexité. Il applique les notions vues au cours de manière structurée : navigation avec Expo Router, formulaires validés avec React Hook Form et Zod, stockage local relationnel avec SQLite, séparation claire entre interface et accès aux données, et prise en compte de l'expérience mobile avec la saisie rapide et le retour haptique.

Dans le cadre d'un examen de troisième année, le projet présente donc une base fonctionnelle complète, cohérente techniquement et défendable oralement.